PORTUNUS · NEW FEATURE

Vault signatures

Server-side tamper detection for the Drive vault file

The server signs every save with Ed25519 and checks the signature on every load. A file that was changed outside Portunus is rejected before the browser ever tries to unlock it.

Ed25519 | Server-only | No client crypto added | Migration flag for existing vaults

WHY

The problem it solves

The vault lives in your Google Drive as `portunus.vault.json`. Portunus is not the only thing that can write to it.

Who can change the file

- Anyone with access to your Google account or Drive
- A Drive sync client on any of your machines
- Any other tool holding a Drive token for the file

What we already had

- Client-side XChaCha20-Poly1305 with Argon2id
- The vault header is bound into the AEAD (as AAD)
- A tampered ciphertext already fails to decrypt

What signatures add

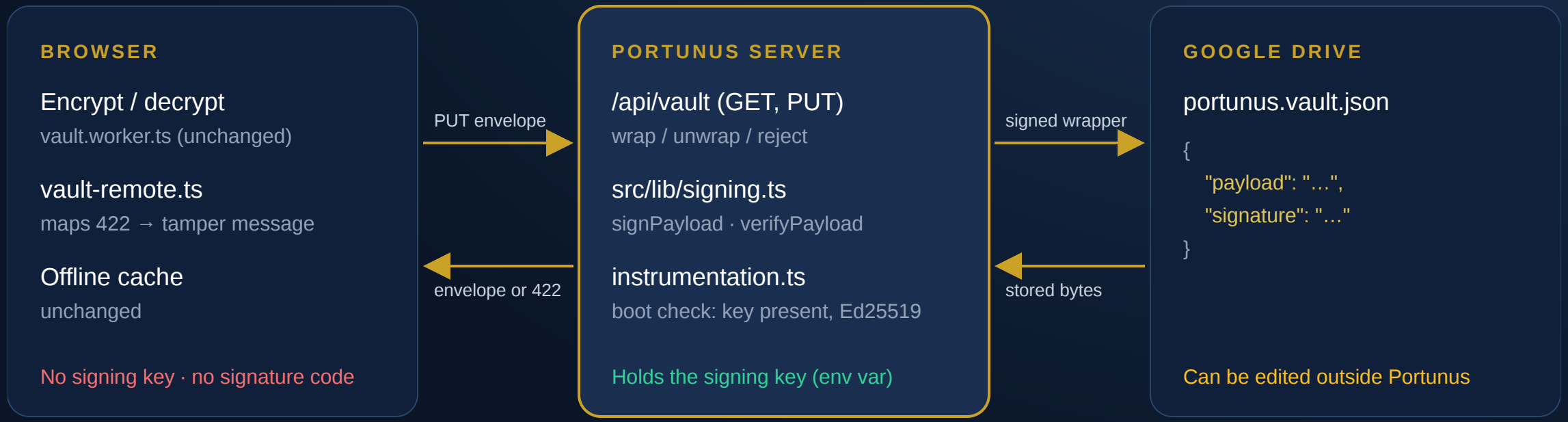
- An explicit server-side check of the **whole** file
- Done **before** anything reaches the browser
- A clear error ("may have been tampered with") instead of a confusing decrypt failure

> Zero-knowledge is unchanged: the server signs and verifies opaque ciphertext and never sees plaintext or the master password.

ARCHITECTURE

Who does what

The signing key exists only on the server. The browser never sees the signature wrapper.



STORAGE FORMAT

What lands in Drive

The envelope string the client sends is signed byte for byte and nested inside a small wrapper.

Before · legacy bare envelope

```
// portunus.vault.json
{
  "format": "portunus-vault",
  "version": …,
  "engine": "…",
  … KDF params, nonce, ciphertext
}
```

After · signed wrapper

```
// portunus.vault.json
{
  "payload": "{\"format\":\"portunus-vault\", …}",
  "signature": "base64 Ed25519, 64 bytes"
}
```

What gets signed

The UTF-8 bytes of the exact `payload` string. The server never parses or re-serialises it, so the bytes the client sent are the bytes that get verified.

What the client receives

Only `{ fileId, modifiedTime, contents }`, where `contents` is the payload. The wrapper never leaves the server.

FLOW 1 OF 2

Saving the vault · `PUT /api/vault`

Signing is the last step before the Drive write. Validation and the conflict check are unchanged.

1 — Browser encrypts

The worker produces the envelope string. The browser sends it as `contents`, with `fileId` and `baseModifiedTime`.

2 — Validate

Session check, then `contents` must be a non-empty string under the size limit. Otherwise 400.

3 — Conflict check

If Drive's `modifiedTime` ≠ `baseModifiedTime`, return 409. Nothing is written.

4 — Sign

`signPayload(contents)` runs Ed25519 over the UTF-8 bytes and returns base64.

5 — Write to Drive

`updateVault` or `createVault` stores the output of `wrapSigned()`: payload plus signature.

Every save writes the signed format. That is how existing vaults migrate: each one is signed the next time it's saved.

Signatures disabled or demo mode: step 4 is skipped and the bare envelope is stored, as before this feature.

FLOW 2 OF 2

Loading the vault · `GET /api/vault`

After `downloadVault`, `parseStoredVault` sorts the file into one of three kinds, and `openStoredVault` decides.

STORED FILE	SIGNATURES ENABLED	ENABLED + ALLOW_MISSING_SIGNATURE=1	DISABLED / DEMO
Signed wrapper valid signature	200 payload	200 payload	200 payload unwrapped, not verified
Signed wrapper bad signature or edited payload	422 vault_signature_invalid	422 vault_signature_invalid the flag never relaxes this	200 payload unwrapped, not verified
Malformed wrapper has payload/signature key, wrong shape	422	422 never counts as legacy	200 verbatim
Legacy bare envelope no signature at all	422	200 verbatim logs “accepted unsigned vault”	200 verbatim

A 422 response carries only `{ "error": "vault_signature_invalid" }`, with no vault contents. Every rejection is logged server-side with the file ID.

USER EXPERIENCE

What the user sees on a tampered file

The only client change is recognising one new error.

Vault signature invalid — the file may have been tampered with. Not unlocking.

- `isVaultSignatureInvalid(e)` matches status 422 plus the error code `vault_signature_invalid`
- Shown on the first vault fetch (`VaultApp`) and when unlocking (`VaultUnlock`)
- Unlock is refused; no decryption is attempted

Deliberately unchanged

- **Offline cache.** Only a network `TypeError` falls back to the cached ciphertext. A 422 is an HTTP response, so it is surfaced, not masked.
- **Worker.** `vault.worker.ts` has no new crypto.
- **Encryption.** Envelope format, KDF and cipher are untouched.

CONFIGURATION

Environment variables

All are read on the server only. Signatures are **on by default**.

VARIABLE	EFFECT	KEY REQUIRED?
`PORTUNUS_SIGNING_PRIVATE_KEY`	Ed25519 private key: base64 of the DER PKCS#8 key, on one line. The public key is derived from it, so there is no second variable.	—
`PORTUNUS_ALLOW_MISSING_SIGNATURE=1`	Migration only: accepts legacy unsigned files on GET. A present but invalid signature still fails.	Yes saves still sign
`PORTUNUS_SIGNATURES_DISABLED=1`	Turns the feature off: saves are bare envelopes, and signed files are unwrapped without checking.	No
`PORTUNUS_DEMO_MODE=1`	Demo mode skips signing the same way.	No

Breaking for existing self-hosters: after upgrading, the server won't start unless a key is set, `PORTUNUS_SIGNATURES_DISABLED=1` is set, or demo mode is on.

STARTUP

Fail fast at boot

`instrumentation.ts` → `checkRequirements()` → `assertSigningRequirements()`

```ts
// src/instrumentation.ts
export async function register() {
  if (process.env.NEXT_PHASE === "phase-production-build")
    return;
  if (process.env.NEXT_RUNTIME === "nodejs") {
    const { checkRequirements } =
      await import("@/lib/check-requirements");
    checkRequirements();
    setupOpenTelemetry();
  }
}
```

The dynamic import keeps `node:crypto` out of the edge bundle. A static import broke `next start` with "Native module not found: node:crypto".

Startup errors name the variable

- `PORTUNUS_SIGNING_PRIVATE_KEY is not set…` (the message points to the README and the disable flag)
- `…must be base64 DER (PKCS#8), not PEM`, followed by the conversion command
- `…is not valid base64`
- `…is not a valid PKCS#8 private key`
- `…must be an Ed25519 key, got rsa`

Keys are parsed once and cached per raw env value.

SETUP

Generating the key

Once per deployment. Treat it as being as secret as `AUTH_SECRET`.

```
# 1. Create an Ed25519 private key (PEM, PKCS#8)
openssl genpkey -algorithm ed25519 -out portunus-signing.pem

# 2. Convert it to the env var format: base64 of the DER PKCS#8 key, one line
openssl pkey -in portunus-signing.pem -outform DER | base64 | tr -d '\n'
MC4CAQAwBQYDK2VwBCIEI…   # 64 characters

# 3. Set it (.env, docker-compose.yml, or your secret manager)
PORTUNUS_SIGNING_PRIVATE_KEY=MC4CAQAwBQYDK2VwBCIEI…
```

Back up the key. A signed vault can only be verified with the key that signed it.

A PEM value is rejected at startup, and the error message includes the conversion command.

ROLLOUT

Migrating an existing install

Vaults saved before this feature have no signature. They migrate as they are re-saved.

1

Deploy with the migration flag

Set `PORTUNUS_SIGNING_PRIVATE_KEY` **and** `PORTUNUS_ALLOW_MISSING_SIGNATURE=1`, then restart.

›

2

Re-save each vault

Each user unlocks, makes any change and saves. The save writes the signed format. Until then the server logs `accepted unsigned vault …`

›

3

Watch the logs

When the "accepted unsigned vault" warnings stop, every vault is signed.

›

4

Enforce

Remove `PORTUNUS_ALLOW_MISSING_SIGNATURE` and restart. An unsigned file now returns 422.

Migration-window risk: while the flag is set, an unsigned file is accepted. That includes a tampered file whose signature was stripped back to a bare envelope. Keep the window short. A file that *has* a signature is always verified, flag or not.

TRUST MODEL

What it protects, and what it doesn't

Signing and verification happen only on the server, with a key only the server holds.

Detected

- Edits to `portunus.vault.json` made outside Portunus: in the Drive UI, by a sync client or by another tool
- A signature that was swapped, truncated or corrupted
- A wrapper that was half-stripped (for example, the signature field removed)
- A stripped-back bare envelope, once the migration flag is removed

Not covered

- **A compromised Portunus server.** It holds the key and serves the client code.
- **Rollback.** The signature covers only the payload, so someone with Drive access can put back an *older* file this server signed.
- **Unsigned files during migration**, while `ALLOW_MISSING_SIGNATURE` is set.

Design note: a save doesn't re-verify the file it overwrites. The `modifiedTime` conflict check already blocks a save after an outside edit, and in the rare case it misses one, the tampered bytes are overwritten, never served.

Key rotation and test coverage

Rotating or replacing the key

Every signed vault fails verification under a different key. To rotate:

1. Set `PORTUNUS_SIGNATURES_DISABLED=1` and re-save every vault
2. Remove that flag and set the new key
3. Also set `ALLOW_MISSING_SIGNATURE=1` until every vault has been saved again, because saves made while disabled are unsigned
4. Remove the migration flag

Tests (Vitest)

- `signing.test.ts`: round trip, edited payload, wrong key, malformed signatures, disabled and demo passthrough, boot failures (missing, PEM, garbage, non-Ed25519)
- `route.signing.test.ts`: GET with signed, tampered, malformed and unsigned files (flag on and off, disabled, demo), plus signed vs. bare PUT
- `check-requirements.test.ts`: boot fails without a key
- `vault-remote.test.ts`: 422 error-code mapping
- Drive round-trip integration test runs with a real key